Harshit Daheriya

es23btech11017@iith.ac.in | es23btech11017@iith.ac.in | +91 8629958243 | LinkedIn | GitHub

EDUCATION

Institution	Degree / Board	Passing Year	Score / CGPA
Indian Institute of Technology Hyderabad	B.Tech	2027	8.66
Saint Norbert Higher Secondary School	Class XII (CBSE)	2023	93.00%
Saint Norbert Higher Secondary School	Class X (CBSE)	2021	92.00%

WORK EXPERIENCE

Morgan Stanley | Software Engineering Summer Analyst | Bangalore | May - Jul 2026

- **Scaled the in-house email engine from ~150 to 1,000+ emails/sec** by redesigning it from a synchronous, connection-per-email sender into an asynchronous Kafka + Akka Streams pipeline, exceeding the target in load testing.
- **Rebuilt the send path for throughput**: isolated blocking sends from the Akka Streams threads so they could no longer stall the stream, and replaced per-email connection setup with a pool of reusable mail-server connections.
- **Made the pipeline robust and well-tested**: applied Akka Streams backpressure to eliminate downstream message dropping under load spikes, and covered the redesigned pipeline with unit tests at 90% code coverage.

PROJECTS

Kites -- RISC-V Simulator & Visual Debugging Environment

C++20 | *Qt6* | *Lua* | *GitHub*

- **Extended a single-stage RISC-V VM into a 5-stage pipelined RV32/RV64 simulator**, implementing data-hazard detection, operand forwarding across all producer/consumer stage pairs, load-use stalls and **real-time pipeline visualization**.
- **Designed a cache simulation framework** with L1/L2 levels and configurable write, allocation and replacement policies.
- **Exposed a Lua scripting API** for user-defined replacement policies at runtime, without recompiling the C++ core.
- **Integrated debugging and profiling**: breakpoints, stepping, register/memory inspection and an execution heat map.
- **Wrote regression tests for the pipelined execution**, run automatically through the project's CI pipeline.

Intelligent Timetable Management System

TypeScript | *React* | *Express* | *MongoDB* | *Course project, team of 6*

- Web platform that assigns courses to rooms and time slots using an OR-Tools CP-SAT solver.
- **Built student and faculty dashboards** on Express/TypeScript APIs for courses, exams and notifications.
- **Built the initial faculty class-rescheduling workflow**, from request models and Zod-validated APIs to the UI.
- **Added an LLM timetable assistant** grounding Gemini answers in each student's enrollments and weekly schedule.

Per-User Fairness in Flow-Aware Networks

Python | *Ryu (OpenFlow 1.3)* | *Mininet* | *Team project*

- **Implemented Flow-Aware Networking with per-user fairness** as a Ryu OpenFlow 1.3 controller, reproducing Doma et al. (ICC 2012): measurement-based admission control capping admitted TCP flows per user in each 1 s window.
- **Estimated the fair rate from OpenFlow flow statistics** with EWMA smoothing; under congestion, evicted flows exceeding 1.2× the fair rate (RAMAF) and readmitted them FIFO from a priority-access list once congestion cleared.
- **Evaluated on a Mininet testbed** with a 100 Mbps CAKE bottleneck and a lossy 5 Mbps wireless link: the lossy-link user reached over **2× the throughput** of a per-flow FAN baseline at **0.01-1% loss** under high load.

MiniML Interpreter: Exception Handling Extension

OCaml | *Menhir* | *OCamllex*

- **Added raise and try...with to MiniML's grammar** in Menhir/OCamllex, with new AST nodes for both constructs.

- **Extended the type checker** with typing rules for raise and try...with, unifying handler and body result types.
- **Implemented exception propagation in the evaluator**, unwinding call frames to the nearest matching handler.

Custom Programming Language Interpreter

Java

- **Built a tree-walking interpreter** in Java for a custom object-oriented language, with no external libraries.
- **Hand-wrote the lexer, parser and AST** without parser generators, with syntax and runtime error reporting.
- **Implemented classes with inheritance**, user-defined functions, arrays, conditionals and loops.

SKILLS

Languages: C++17/20, C, Java, OCaml, Python, TypeScript

Systems & Tools: Git, Linux, CMake, CI (GitHub Actions), Mininet

Frameworks & Libraries: Apache Kafka, Akka Streams, Qt6, Lua (C++ embedding), Ryu (OpenFlow), React, Express, MongoDB

Concepts: Data Structures & Algorithms, Computer Architecture, Operating Systems, Object-Oriented & Functional Programming

RELEVANT COURCES

Computer Science: Data Structures and Applications, Algorithms, Operating Systems-I, Computer Architecture, Theory of Computation, Database Management Systems-I, Software Engineering, Software Development Fundamentals

POSITIONS OF RESPONSIBILITY

Public Relations & Outreach Coordinator, Tinkerers' Lab

Operations Manager, E-Cell

Sunshine UG Buddy

Head of Logistics, X-MUN